EXP NO: 9 MODEL DEPLOYMENT: REST API WITH FLASK AND CONTAINERIZATION WITH DOCKER

AIM

To deploy a machine learning model as a REST API using Flask and package the application using Docker for consistent execution.

ALGORITHM

1. Import Flask and the required Scikit-learn libraries.
2. Load the Iris dataset and train a classification model.
3. Create a Flask application and define a POST endpoint /predict.
4. Receive input features as JSON and validate the input.
5. Use the trained model to predict the flower class and return JSON output.
6. Create requirements.txt and a Dockerfile.
7. Build a Docker image and run the container with port mapping.
8. Send a sample request and verify the prediction response.

CODE: app.py

```
from flask import Flask, request, jsonify
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier

app = Flask(__name__)

iris = load_iris()
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(iris.data, iris.target)

@app.route("/", methods=["GET"])
def home():
    return jsonify({"message": "Iris prediction API is running"})

@app.route("/predict", methods=["POST"])
def predict():
    data = request.get_json(silent=True)
    if not data or "features" not in data:
        return jsonify({"error": "Send JSON with a features list"}), 400

    features = data["features"]
    if not isinstance(features, list) or len(features) != 4:
        return jsonify({"error": "Provide exactly 4 numeric features"}), 400

    try:
        values = [float(x) for x in features]
        prediction = int(model.predict([values])[0])
        return jsonify({
            "prediction": prediction,
            "class_name": iris.target_names[prediction].item()
        })
    except (TypeError, ValueError):
        return jsonify({"error": "Features must be numeric"}), 400

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

CODE: requirements.txt

```
Flask
scikit-learn
gunicorn
```

DOCKER CONFIGURATION

CODE: Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "app:app"]
```

COMMANDS

```
# Install dependencies and run locally
pip install -r requirements.txt
python app.py

# Build the Docker image
docker build -t iris-api .

# Run the container
docker run -p 5000:5000 iris-api
```

TEST THE API

```
POST http://127.0.0.1:5000/predict
Content-Type: application/json

{
  "features": [5.1, 3.5, 1.4, 0.2]
}
```

SAMPLE OUTPUT

```
{
  "prediction": 0,
  "class_name": "setosa"
}
```

RESULT

The machine learning model was successfully deployed as a REST API using Flask. The application accepts JSON input and returns predictions in JSON format. Docker was used to package the application and its dependencies for consistent deployment.